

Sistemas de Big Data

Actividad Fase 3:



SmartChef

Curso de Especialización en Inteligencia Artificial y Big Data
MODALIDAD ONLINE

4 de mayo de 2026

Grupo 9:

- | | |
|-------------------------------|---------------------------|
| · Alejandro Fernández Fuentes | - Data |
| · Antoni Sureda Cardona | - Platform |
| · Blas Martos Ortega | - Machine Learning |
| · Hugo Barrera Castillo | - Project Management / BI |

[Repositorio Github - Smartchef](#)

[Video SMARTCHEF presentación final.](#)

Índice

1. Estado Final del Producto	3
1.1. Funcionalidades Completas	3
1.2. Mejoras realizadas respecto a la Fase 2	3
1.3. Qué se ha optimizado	3
1.4. Qué se ha automatizado	4
1.5. Qué sigue siendo limitación	4
2. Mejoras implementadas	5
2.1. Frontend (UI/UX)	5
2.2. Data Pipeline y Machine Learning	5
2.3. Backend y Lógica de Negocio (API)	5
2.4. Infraestructura y Despliegue	6
2.5. Seguridad y Persistencia	6
3. Automatización	7
3.1. Arranque del proyecto con un solo comando	7
3.2. Scripts creados y Orquestación (Docker / Compose)	7
4. Despliegue	8
5. MLOps	9
6. Arquitectura final	10
6.1. Componentes finales	10
6.2. Automatizaciones y Despliegue	11
7. Documentación final	11



1. Estado Final del Producto

El producto final, **SmartChef**, se consolida como una solución analítica y predictiva *end-to-end* orientada a la optimización operativa y reducción del desperdicio alimentario en el sector de la restauración. A continuación, se detalla el estado definitivo del sistema:

1.1. Funcionalidades Completas

El sistema ofrece un Dashboard interactivo, desarrollado en Vanilla JS, ágil y estructurado en cinco módulos clave:

- **Resumen Ejecutivo:** Monitorización de KPIs operativos cruzados con variables de contexto externo (temperatura media, días de lluvia y festividades) detectados automáticamente.
- **Ventas y Categorías:** Visualización de la rentabilidad del menú y la proporción de éxito relativo de cada plato mediante gráficos históricos.
- **Predicciones:** Estimación precisa de la necesidad de ingredientes brutos a comprar, respaldada por un Intervalo de Confianza estadístico para gestionar la volatilidad e incertidumbre.
- **Riesgo de Desperdicio:** Detección automática de anomalías y sobre-stock en ingredientes perecederos, alertando de desviaciones predictivas frente al consumo promedio.
- **Ingredientes:** Auditoría y ranking del consumo de materias primas, unificando los datos en magnitudes de compra reales.

1.2. Mejoras realizadas respecto a la Fase 2

Durante la iteración final, se ha elevado la madurez del proyecto aplicando mejoras críticas en todas sus capas:

- **Frontend (UI/UX):** Integración de indicadores de carga para el asincronismo y bloques *try-catch* para gestionar caídas de red con alertas visuales. Eliminación de columnas redundantes para centrar la carga cognitiva.
- **Machine Learning:** Se corrigió la tendencia a sobreestimar la demanda separando estrictamente los datos de entrenamiento y validación de forma cronológica. El modelo *Random Forest* ha sido regularizado (limitando profundidad y muestras por hoja) y evaluado mediante la métrica MAE. Además, las predicciones se exportan en CSV versionados para garantizar la trazabilidad.
- **Backend:** Implementación de un manejo robusto de excepciones. La API ahora utiliza bloques *try-except* y devuelve códigos HTTP precisos (404, 500), garantizando un cierre controlado de las conexiones a la base de datos tras cada consulta.
- **Infraestructura y Seguridad:** Migración a una arquitectura basada en contenedores con **Docker Compose**. Se ha incorporado **Nginx** como proxy inverso configurado con SSL local, resolviendo nativamente los bloqueos por CORS y enmascarando las peticiones bajo el dominio principal (/api/). Adicionalmente, se ha integrado la gestión segura de secretos mediante un archivo `.env`.

1.3. Qué se ha optimizado

- **Rendimiento y Delegación de Carga:** Se ha optimizado la arquitectura delegando a Nginx la entrega exclusiva de recursos estáticos (HTML/CSS/JS). Esto libera a FastAPI de tareas de servidor web, permitiendo que dedique toda su capacidad computacional a resolver la lógica pesada y los modelos de IA.
- **Legibilidad y Visualización de Datos:** Refactorización dinámica del DOM y gráficas de *Chart.js*, limitando la visualización al "Top 15" de ingredientes. Esto resuelve los problemas previos de solapamiento en los ejes y mejora drásticamente la legibilidad.
- **Normalización Estructural:** El backend estandariza las métricas de las recetas y entrega todas las cantidades en unidades de medida paramétricas (kg, ud, litros), facilitando el inventariado sin requerir conversiones manuales.
- **Consultas Dinámicas:** Refactorización del código SQL para eliminar las fechas dependientes, haciendo que las extracciones de datos se adapten siempre a los últimos registros disponibles.

1.4. Qué se ha automatizado

- **Despliegue robusto One-Click:** Orquestación total mediante el script `start.sh` con *healthchecks* dinámicos. El sistema autogestiona su orden de arranque: el contenedor de backend permanece en espera activa y solo arranca cuando PostgreSQL está al 100%, evitando fallos de sincronización iniciales.
- **Backfilling Temporal:** Implementación de una lógica de sincronización de fechas que detecta automáticamente los "huecos" en los que el sistema ha estado apagado, rellenando el histórico sin intervención manual para no quebrar las series temporales.
- **Generación de Alertas de Contexto:** El sistema cruza promedios históricos con eventos externos de forma autónoma. Si detecta condiciones (ej. lluvias) que históricamente alteran el consumo de un plato específico, automatiza la alerta predictiva de desviación en el cuadro de mando.

1.5. Qué sigue siendo limitación

A pesar de la madurez operativa del sistema, se reconocen las siguientes limitaciones y áreas de trabajo futuro:

- **Dependencia de la volumetría de datos:** El modelo predictivo requiere de un histórico sustancial para afinar su precisión. Para platos nuevos, de temporada o con muy baja rotación, el nivel de incertidumbre aumenta, provocando un ensanchamiento inevitable del Intervalo de Confianza.
- **Falta de integración en Tiempo Real:** Actualmente, la ingesta de datos operativos se basa en volcados estáticos. SmartChef carece de una conexión bidireccional nativa con un software TPV comercial para la actualización de consumos al minuto.
- **Credenciales estáticas para el MVP:** Aunque la arquitectura soporta plenamente la inyección dinámica de secretos, la versión entregada requiere el uso de credenciales de demostración predefinidas en el archivo local `.env`. Esto es estrictamente funcional para facilitar una evaluación académica sin fricciones, pero requeriría la reconfiguración de un gestor de secretos (como AWS Secrets Manager) en un entorno de producción público.

2. Mejoras implementadas

A lo largo de esta fase final, el proyecto ha evolucionado de un entorno de desarrollo local a una arquitectura robusta, segura y automatizada. Las mejoras implementadas por el equipo se dividen en las siguientes:

2.1. Frontend (UI/UX)

- **Gestión de estados y feedback:** Se han incorporado indicadores visuales de carga (*spinners*) durante el procesamiento de las llamadas asíncronas a la API, mejorando la respuesta interactiva del Dashboard.
- **Manejo de excepciones:** Implementación de bloques *try-catch* para gestionar caídas del backend o errores de red, renderizando alertas visuales explícitas para el usuario.
- **Optimización de la visualización:** Refactorización de las gráficas para limitar la visualización al "Top 15" de ingredientes, resolviendo problemas de solapamiento y mejorando la legibilidad.
- **Limpieza de la interfaz:** Eliminación de columnas redundantes en las tablas de clasificación para centrar la atención en los datos predictivos clave.
- **Normalización y corrección de unidades de medida:** Ajuste en el renderizado del DOM para garantizar que cada ingrediente muestre su unidad paramétrica correspondiente (kg, ud, litros) tanto en la Matriz de Riesgo de Desperdicio como en los gráficos. Esto corrige discrepancias visuales y facilita un inventariado preciso para la toma de decisiones.

2.2. Data Pipeline y Machine Learning

- **Automatización de flujos temporales:** Se ha implementado una lógica de sincronización de fechas dinámica que permite al sistema detectar los "huecos" de días en los que el sistema ha estado apagado, ejecutando un *backfilling* automático para no perder continuidad en los datos.
- **Enriquecimiento del contexto:** Integración con APIs externas para incluir variables exógenas (como el clima) directamente en el proceso de ingesta.
- **[Nota para tu compañero: Añade aquí 1 o 2 puntos sobre cómo has mejorado el modelo, si has cambiado hiperparámetros, o cómo optimizaste el script de entrenamiento]**

2.3. Backend y Lógica de Negocio (API)

- **Resolución de dependencias dinámicas:** Refactorización de las consultas SQL en el endpoint principal para que la extracción de datos no dependa de filtros de fecha *hardcodeados*, adaptándose automáticamente a los últimos registros disponibles en la base de datos.

- **Consolidación de Endpoints:** Reestructuración de las rutas de FastAPI para servir la información de inventario, riesgo y predicciones en estructuras JSON más eficientes para el consumo del Frontend.

2.4. Infraestructura y Despliegue

- **Orquestación con contenedores:** Migración a una arquitectura de microservicios mediante Docker Compose, aislando Frontend, Backend, Base de Datos y Pipeline en entornos reproducibles.
- **Despliegue "One-Click":** Desarrollo del script **start.sh** con mecanismos de *healthcheck* dinámicos (pg_isready). El sistema ahora gestiona su propio orden de arranque, esperando a que la base de datos esté plenamente operativa antes de lanzar las ingestas de datos.

2.5. Seguridad y Persistencia

- **Comunicaciones Seguras (HTTPS):** Generación de certificados SSL locales para asegurar que el tráfico del Dashboard viaje cifrado a través del proxy inverso de Nginx.
- **Gestión de Secretos:** Eliminación de credenciales en texto plano. Se ha implementado la inyección de variables de entorno mediante un archivo **.env** excluido del control de versiones. Si bien para esta entrega se proporcionan las credenciales de PostgreSQL predefinidas a fin de asegurar un despliegue local exitoso y auditable.
- **Mitigación CORS y Aislamiento:** Nginx enmascara las peticiones bajo el mismo origen (/api/) para evitar bloqueos del navegador. Además, la base de datos PostgreSQL se ha aislado en una red virtual privada, garantizando la persistencia mediante volúmenes de Docker (db_data).

2.6 Ajuste y mejora del modelo de Machine Learning

- **Detección del problema:** Durante el análisis de los resultados obtenidos en la Fase 2, se observó que el modelo de Random Forest tendía a sobreestimar la demanda futura, especialmente en determinados platos y escenarios de contexto. Este comportamiento es habitual en modelos entrenados con todo el histórico disponible sin validación adecuada.
- **Corrección del sesgo optimista**
 - Separación temporal del conjunto de datos, dividiendo el histórico en datos de entrenamiento y validación sin barajar el orden cronológico.
 - Evaluación cuantitativa del modelo mediante la métrica MAE (Mean Absolute Error), permitiendo medir el error medio de las predicciones.
 - Regularización del modelo Random Forest, limitando la profundidad máxima de los árboles y el número mínimo de muestras por hoja, con el objetivo de reducir el sobreajuste y suavizar las predicciones extremas.
- **Versionado de resultados del modelo:** Como parte de las buenas prácticas de Machine Learning, las predicciones generadas se exportan a archivos CSV versionados por fecha de ejecución, lo que permite:
 - Evitar la sobrescritura de resultados anteriores

- Mantener trazabilidad entre ejecuciones del modelo.
- Facilitar la comparación de predicciones a lo largo del tiempo.

2.7 Seguridad y gestión de errores en la API (FastAPI)

En fases anteriores, la API devolvía respuestas genéricas incluso en caso de error, lo que dificultaba la detección de fallos y no cumplía con las buenas prácticas REST.

- **Mejora del manejo de errores:** Durante la Fase 3 se reforzó la API implementada con FastAPI mediante:
 - Uso explícito de bloques try / except.
 - Lanzamiento de excepciones HTTP con códigos de estado adecuados:
 - 404 cuando no existen datos disponibles.
 - 500 en caso de errores internos del servidor.
 - Cierre controlado de conexiones a la base de datos.

Estas mejoras permiten que el cliente reciba respuestas claras y coherentes ante cualquier situación de error.

3. Automatización

3.1. Arranque del proyecto con un solo comando

Para garantizar la reproducibilidad y facilitar la evaluación del entorno, el despliegue de toda la infraestructura se ha automatizado mediante un único comando de terminal. Ejecutando el script **./start.sh** desde el directorio raíz, el sistema gestiona de forma autónoma el ciclo de vida completo de la aplicación, sin necesidad de intervención manual por parte del usuario.

3.2. Scripts creados y Orquestación (Docker / Compose)

El proyecto está orquestado íntegramente bajo **Docker Compose**, definiendo los servicios, redes y volúmenes en el archivo `docker-compose.yml`. Para automatizar tanto el despliegue como el flujo de datos, se han creado los siguientes scripts:

- **start.sh:** Este script principal realiza las siguientes tareas secuenciales:
 1. **Limpieza de entorno:** Ejecuta `docker compose down` para eliminar contenedores huérfanos o ejecuciones previas, asegurando un inicio limpio.
 2. **Construcción y Despliegue:** Lanza `docker compose up -d --build`, procesando los archivos `.dockerignore` para optimizar el peso de las imágenes y levantando los contenedores en segundo plano.
 3. **Sincronización mediante Healthchecks Dinámicos:** Implementa un bucle condicional (until) apoyado en el comando `pg_isready`. Este realiza comprobaciones continuas al contenedor de PostgreSQL y pausa la ejecución del resto del sistema hasta que la base de datos confirma que está lista para aceptar conexiones, evitando errores de desconexión prematura.
- **run_pipeline.sh:** Una vez la base de datos está plenamente operativa, se ejecuta este script dentro de su propio contenedor aislado. Automatiza el flujo ETL

(Extracción, Transformación y Carga) y el ciclo de Machine Learning en cuatro fases críticas:

1. **Actualización de datos en crudo y Contexto:** Dispara secuencialmente la generación de históricos (`generate_raw_historic.py`), la ingesta en base de datos (`ingest.py`) y la actualización de variables exógenas como el clima o festivos (`context_update.py`).
2. **Sincronización de Base de Datos:** Ejecuta `db_sync.py` para consolidar y preparar la información, garantizando que el modelo reciba datos íntegros y estructurados.
3. **Entrenamiento e Inferencia:** Lanza `train_predictions.py` para entrenar el modelo predictivo utilizando el dataset recién actualizado y generar las nuevas estimaciones de demanda de ingredientes.
4. **Disponibilidad de Predicciones:** Finalmente, `import_predictions.py` vuelca de forma automática los resultados generados en la tabla `fact_predictions`, dejándolos inmediatamente disponibles para su consumo a través de la API REST y su visualización en el Dashboard del usuario.

4. Despliegue

Justificación técnica de no despliegue en Cloud De acuerdo con los requisitos del proyecto, se aporta a continuación la justificación técnica por la cual no se ha realizado un despliegue en la nube. Aunque la arquitectura basada en contenedores y microservicios es inherentemente compatible con entornos Cloud, se ha tomado la decisión estratégica de mantener el proyecto como un despliegue local automatizado por los siguientes motivos:

1. **Altos requisitos computacionales de los procesos de Machine Learning:** Los flujos de trabajo relacionados con Inteligencia Artificial, específicamente la ingesta masiva de datos y el entrenamiento de modelos predictivos, requieren un alto consumo de memoria RAM y capacidad de procesamiento. Las capas gratuitas que ofrecen los proveedores Cloud estándar limitan drásticamente estos recursos, lo que habría provocado inestabilidad en el sistema y bloqueos por falta de memoria. Escalar a servidores con las prestaciones adecuadas suponía un coste económico no justificado para la validación de un Producto Mínimo Viable académico.
2. **Garantía de auditoría y reproducibilidad absoluta:** El objetivo principal en esta fase es demostrar la solidez de la orquestación y la automatización del flujo de datos. Al entregar el ecosistema completamente encapsulado y con scripts de arranque desatendidos, se garantiza que el equipo evaluador pueda reproducir la infraestructura de manera idéntica en cualquier equipo. Esto elimina el riesgo de problemas externos que podrían entorpecer la corrección, como caídas temporales del proveedor Cloud, restricciones de red o la caducidad de cuentas de prueba.

3. **Seguridad y enfoque del MVP:** Exponer una arquitectura de datos completa (que incluye bases de datos relacionales, APIs y pipelines de ingesta) a la internet pública exige la implementación de capas complejas de ciberseguridad, gestión de identidades y control de acceso. Dado que el alcance principal del proyecto es demostrar la viabilidad del modelo predictivo y la correcta estructuración del *Big Data*, mantener el despliegue restringido a una red virtual privada y aislada localmente minimiza la superficie de ataque y permite concentrar los esfuerzos de desarrollo en el valor central de la aplicación.

5. MLOps

El proyecto SmartChef incorpora una aproximación básica pero coherente a prácticas de **MLOps**, adaptadas al contexto académico y al alcance definido del sistema.

El modelo de Machine Learning no se reentrena de forma automática ni en tiempo real. Esta decisión se ha tomado de manera consciente, ya que el proyecto trabaja con un conjunto de datos históricos estable y no con flujos de datos en streaming o entornos productivos con cambios constantes.

El **versionado del modelo** se gestiona indirectamente mediante el control de versiones del código fuente (Git), de forma que cada modificación en el proceso de entrenamiento queda registrada y asociada a una versión concreta del repositorio.

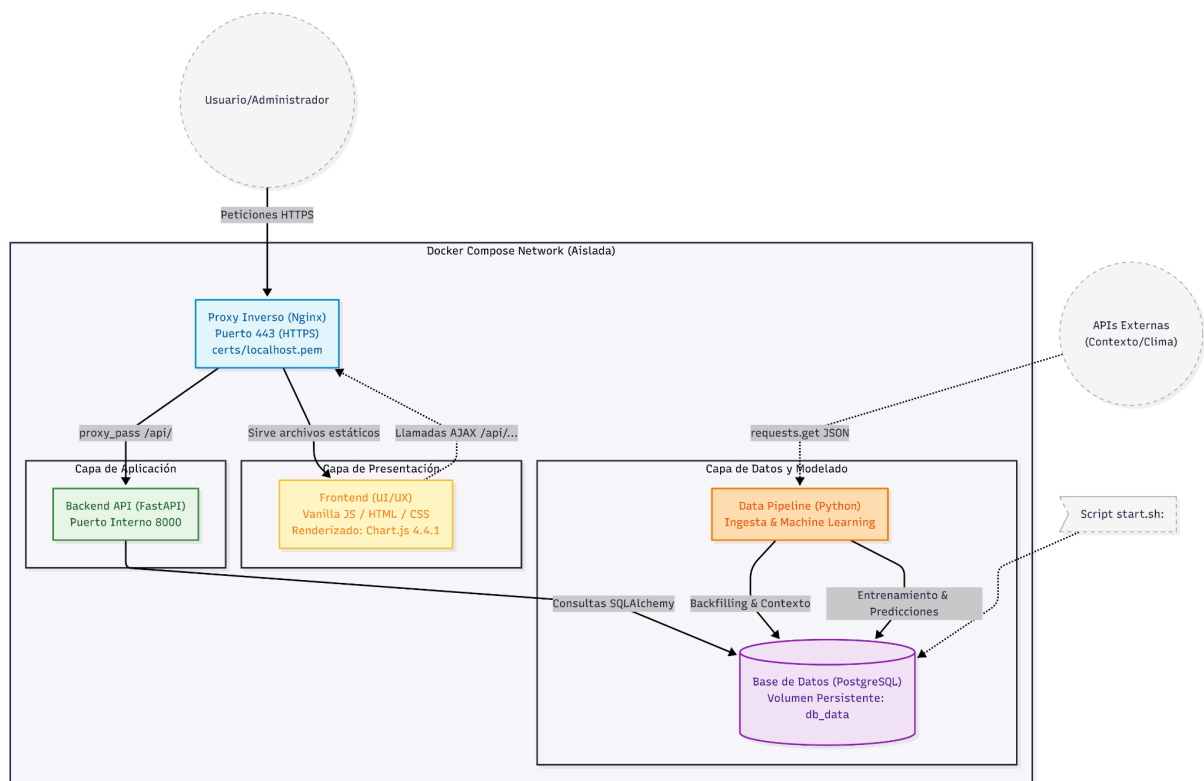
Adicionalmente, los resultados del modelo se exportan a **ficheros CSV versionados por fecha de ejecución**, lo que permite:

- Mantener trazabilidad entre ejecuciones del modelo.
- Evitar la sobrescritura de predicciones anteriores.
- Facilitar el análisis comparativo de resultados a lo largo del tiempo.

El entrenamiento del modelo se ejecuta **bajo demanda**, permitiendo regenerar predicciones cuando se detectan cambios relevantes en los datos o mejoras en el proceso de entrenamiento.

En un entorno productivo, este enfoque podría ampliarse con mecanismos de reentrenamiento periódico, monitorización automática del rendimiento del modelo y detección de degradación de las predicciones. No obstante, para el contexto del proyecto, la solución implementada resulta adecuada y consistente con los objetivos planteados.

6. Arquitectura final



6.1. Componentes finales

La infraestructura se divide en servicios independientes e interconectados dentro de una red virtual privada generada por Docker Compose:

- **Capa de Seguridad y Enrutamiento:** Ejerce de proxy inverso y punto de entrada principal (puerto 443). Centraliza la terminación SSL mediante certificados locales (localhost.pem) y despacha las peticiones. Sirve directamente los archivos estáticos del Frontend y actúa como pasarela (proxy_pass) hacia la API, mitigando de forma nativa los conflictos de CORS en el navegador.
- **Capa de Presentación:** Interfaz de usuario servida de forma segura. Utiliza peticiones asíncronas para consumir la API y renderiza las visualizaciones de datos apoyándose en la librería Chart.js 4.4.1.
- **Capa de Aplicación:** Expuesta en el puerto local 8000 para facilitar la interacción con la documentación de Swagger UI. Procesa la lógica de negocio, atiende las llamadas del dashboard y actúa como interfaz de lectura hacia la base de datos mediante SQLAlchemy.
- **Capa de Datos y Persistencia:** Almacenamiento relacional estructurado. La persistencia de los datos históricos, inventario y predicciones ante paradas del sistema está garantizada mediante el mapeo del volumen físico db_data.
- **Data Pipeline e Ingesta:** Contenedor ETL que opera en segundo plano. Se conecta a APIs externas para obtener variables exógenas (clima, contexto) mediante la librería requests y vuelca los datos procesados e inferencias del modelo predictivo directamente en PostgreSQL.

6.2. Automatizaciones y Despliegue

El despliegue del entorno se ha diseñado para ser totalmente desatendido. La orquestación recae sobre el script de automatización `start.sh`, el cual gestiona el ciclo de vida de la infraestructura:

1. Aplica políticas de limpieza previa eliminando contenedores huérfanos.
2. Despliega la red aislada de Docker Compose, exponiendo únicamente el proxy Nginx al exterior.
3. Implementa un mecanismo de sincronización condicional utilizando el comando `pg_isready`. Esto impone una espera activa que retiene la ejecución del pipeline de ingesta hasta que el motor de PostgreSQL confirma estar 100% operativo, previniendo fallos críticos de conexión durante el arranque.

7. Documentación final

Toda la documentación exhaustiva del proyecto, incluyendo las guías detalladas de usuario y los manuales técnicos, se encuentra consolidada y disponible en el directorio `/docs` del repositorio. No obstante, con el objetivo de facilitar la labor de evaluación del tribunal y garantizar un acceso directo a la puesta en marcha del sistema, se ha extraído e incluido a continuación la **Guía Definitiva de Ejecución y Despliegue**.

El despliegue de SmartChef se ha automatizado bajo la premisa de un entorno One-Click, asegurando que la infraestructura al completo (Base de Datos, Backend con los modelos predictivos y Frontend) pueda ser levantada y evaluada de forma robusta y sin fricciones.

Requisitos Previos del Sistema

Para garantizar la correcta ejecución del ecosistema, la máquina host del evaluador debe contar con los siguientes elementos instalados:

- Motor de contenedores: Docker Desktop (con Docker Compose integrado).
- Entorno de ejecución: Terminal compatible con Bash (Linux, macOS, o WSL/Git Bash en entornos Windows).
- Puertos libres: Asegurar que los puertos 80 y 443 (para Nginx) y 8000 (para FastAPI) no estén siendo utilizados por otros servicios locales.
- Configuración del Entorno (`.env`)

El proyecto sigue la metodología 12-Factor App, utilizando un archivo .env en la raíz del directorio para inyectar los secretos y credenciales de PostgreSQL en los contenedores.

Nota para el tribunal: Para facilitar la evaluación académica y garantizar una conexión exitosa out-of-the-box, el código entregado ya incluye un archivo .env preconfigurado con credenciales de demostración. En un entorno de producción real, este archivo sería excluido del control de versiones por motivos de seguridad.

Proceso de Arranque

Para iniciar la infraestructura, el usuario debe abrir su terminal, navegar hasta el directorio raíz del proyecto y ejecutar los siguientes comandos:

- `chmod +x start.sh`
- `bash start.sh`

Comportamiento del sistema durante el arranque:

El script levantará los contenedores en modo detached. Gracias a la implementación de un healthcheck dinámico, el usuario observará en los logs que el contenedor del Backend permanecerá en un estado de espera activa. Este no comenzará la ingesta de datos ni habilitará la API hasta que el contenedor de PostgreSQL confirme que la base de datos está 100% inicializada y lista para recibir conexiones.

Accesos a la Aplicación

Una vez que la terminal indique que todos los servicios están en estado Running o Healthy, el sistema estará plenamente operativo. El tribunal puede interactuar con el proyecto a través de los siguientes puntos de acceso desde cualquier navegador web:

Dashboard Interactivo (Frontend):

URL: <https://localhost>

Nota: Al utilizar certificados SSL autofirmados para el desarrollo local, el navegador mostrará una advertencia de seguridad rutinaria. El usuario debe hacer clic en Configuración avanzada y seleccionar Continuar a localhost (inseguro).

Auditoría de la API (Swagger UI):

URL: <http://localhost:8000/docs>

Nota: Este puerto se ha dejado expuesto deliberadamente para que el tribunal pueda interactuar con los endpoints del modelo predictivo y realizar pruebas de carga directamente sobre el backend, saltándose la capa de Nginx.

Parada del Sistema

Para detener la ejecución de SmartChef de forma limpia, liberando los puertos y deteniendo los contenedores sin corromper la base de datos, se debe ejecutar el siguiente comando en la raíz del proyecto:

- `docker-compose down`